SafeMeter Prototype: Detailed Description

SEVA submission | Theme: Surakshit Bharat | Track A: Product or Technology

1. What the prototype physically is

The prototype is a compact enclosure, roughly the size of a small speedometer pod, with an OLED display on the front. Inside are the ESP32 and its sensor modules. It is powered from a 12 V source (a vehicle battery, or a bench supply or battery pack for demos) and connects to two external parts: a hall sensor with a magnet on the wheel, and a cancel button on the front.

For the demo it is mounted on a **test rig**: a bicycle or bike wheel on a stand that can be spun by hand or with a small motor, with the unit fixed rigidly to the frame. The rig can be tilted or dropped (padded) to simulate a crash.

2. Hardware blocks and what each does

Block	Component	Role in the prototype
Brain	ESP32	Reads all sensors, runs the crash logic, drives the display and GSM
Motion	MPU6050	Impact magnitude, vibration and tilt angle
Speed	Hall sensor + magnet	True wheel speed, independent of GPS
Location	NEO-6M GPS	Latitude and longitude, backup speed
Comms	SIM800L GSM	Sends the alert SMS without internet
Interface	OLED, buzzer, button	Shows speed and countdown, sounds the alarm, lets the rider cancel
Power	Two buck converters	One 5 V rail for the ESP32 and sensors, one separate ~4 V rail for the GSM module

3. Wiring plan (suggested pins, adjust to your board)

- **I2C bus (SDA 21, SCL 22):** the MPU6050 (address 0x68) and OLED (usually 0x3C) share the same two wires.
- **GPS:** UART2 (RX 16, TX 17).
- **GSM:** a second UART on free pins, for example 26 and 27.
- **Hall sensor:** an interrupt-capable input pin (for example 34), so each wheel revolution is counted without missing pulses.
- **Button:** an input pin with an internal pull-up (for example 32).
- **Buzzer:** an output pin (for example 25).

Two practical points that commonly break student builds:

1. **Power the SIM800L separately.** It needs about 3.4 to 4.4 V and draws current spikes of up to roughly 2 A when it transmits. Do not power it from the ESP32's 3.3 V pin. Use a dedicated buck converter set to about 4 V and add a large capacitor (around 1000 µF) close to the module. Brown-outs and random resets are the most common failure.
2. **Share ground** across all modules, and use a voltage divider or level shifting on the line from the ESP32 to the SIM800L's RX pin.

4. How each sensor produces usable data

Speed. One magnet on the wheel passes the hall sensor once per revolution. The firmware measures the time between pulses, then computes speed as wheel circumference × 3.6 ÷ time per revolution in seconds (giving km/h). Software debouncing ignores false pulses. Adding two or more magnets improves resolution at low speed. The last 2 seconds of speed values are kept in a rolling buffer so the firmware can compare "before" and "after" an impact.

Impact and tilt. The MPU6050 is read at 100 to 200 Hz with its built-in low-pass filter on. The firmware computes total acceleration magnitude (independent of mounting direction) and estimates tilt with a complementary filter. At power-up, while the unit is still, it records the upright reference.

Location. The GPS provides coordinates continuously and the firmware keeps the last valid fix in memory, so even if satellite lock is briefly lost at the moment of the alert, the most recent location is still sent. Allow time for the first fix when starting outdoors, since a cold start can take a minute or more.

5. Firmware behaviour (the state machine)

1. **Normal:** shows live speed on the OLED, logs sensor data, and watches for an impact above the threshold while moving faster than about 20 km/h.
2. **Impact check:** waits about 1 second and checks whether speed collapsed by more than about 60%. A pothole fails here because the wheel keeps turning.
3. **Post-impact check:** confirms the unit is tilted beyond about 60 degrees for more than 2 seconds (for the bike configuration). Otherwise it returns to normal.
4. **Countdown:** the buzzer sounds and the OLED shows "CRASH DETECTED, press to cancel" with a 15-second timer. Pressing the button cancels and logs a "cancelled" event.
5. **Alert:** if not cancelled, the GSM module sends an SMS containing a map link with the coordinates, speed at impact and time. The event is also pushed to the backend if connectivity is available.

Throughout, the code is non-blocking (timers and interrupts rather than long delays), so the display, GPS parsing and sensor sampling keep working at the same time.

6. Alert output

The SMS contains: crash detected, vehicle ID, time, speed before impact, and a Google Maps link to the last location. The prototype sends this to 2 or 3 preset numbers (the team's own phones for the demo). A copy goes to the backend, which triggers the volunteer app.

7. Software components

Volunteer app (Flutter or web app with Firebase):

- Volunteer registration with name, phone and location.
- A push notification on an alert, showing the map and speed at impact.
- An "Accept" button. The first volunteer to accept becomes the responder and others see "case taken".
- Logs time-to-accept for the pilot data.

Dashboard (simple web page):

- A live incident list with time, location, speed at impact and status (cancelled, alerted, responder assigned).

- A map with markers and, as data accumulates, a heatmap of black spots.
- A summary of response times.

8. The test and demo rig

- **Wheel and stand:** a bicycle wheel (or bike wheel) on a fixed stand with the magnet and hall sensor aligned within a few millimetres.
- **Speed generation:** hand-spin for quick tests, or a small motor driving the wheel for repeatable speeds.
- **Pothole simulation:** drop or knock the rig's frame while the wheel is spinning. Impact occurs but the wheel keeps turning, so the system should stay silent.
- **Hard braking:** stop the wheel gradually. There is no spike, so no alarm.
- **Crash simulation:** knock the frame hard enough to spike the accelerometer, stop the wheel quickly, and tilt the unit past the threshold. The countdown should start.
- **Safety:** use padding and keep hands clear of the wheel. Never test on a moving vehicle with a person on it.

9. What is real and what is simulated

Being clear on this builds trust with judges.

Real and working	Simulated or scaled down
Sensor reading, speed calculation, tilt estimate	Crash forces (bench impacts are far weaker than real crashes)
The three-stage detection logic	Mounting on an actual moving vehicle (unless safely tested at low speed on a bike)
Countdown, cancel and SMS delivery	Large-scale volunteer coverage (pilot of 10 to 15 people)
App notification and dashboard logging	High-g sensing (production upgrade)

10. What the prototype proves

- Wheel speed plus impact plus tilt separates crashes from potholes and braking, shown with a labelled test table.

- An alert reaches phones in seconds without internet.
- The cancel window prevents false emergencies.
- The volunteer flow works end to end, with measured response times.

11. Known limitations to state openly

- SIM800L is a 2G module, so it suits the prototype, but production should use 4G.
- The MPU6050 saturates at ±16 g, so a high-g sensor is the upgrade path.
- The vehicle unit cannot tell whether the rider is actually injured, which is why the cancel window and speed-at-impact data exist.
- GPS needs a clear sky and can take time to lock when first switched on.

12. Build order (so something always works)

1. Display plus speed from the hall sensor.
2. Add the IMU and log raw data.
3. Add GPS.
4. Add the detection state machine and countdown.
5. Add GSM and the SMS alert.
6. Add the backend, volunteer app and dashboard.
7. Enclosure and final demo rehearsal.

Built in this order, even if time runs short the team still has a working speedometer with crash detection and SMS, which is the core of the idea.